

BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

Facultad de Ciencias de la Computación



Estructura de Datos y sus Aplicaciones

Clave: FCCA 008

PROYECTO FINAL

Sistema de Detección de Archivos Duplicados por su Contenido

INTEGRANTES DEL EQUIPO:

SIBAJA NERI DIEGO ARMANDO

TAPIA ROMERO AZUL XIMENA

Primavera 2026

ÍNDICE DE CONTENIDO

- 1. Introducción3
 - 1.1 Planteamiento del Problema3
- 2. Descripción de los Algoritmos Utilizados.....3
 - 2.1 Exploración Recursiva de Directorios (scanner.py)3
 - 2.2 Pipeline de Deduplicación en 3 Etapas (detector.py)5
 - 2.2.1 Etapa 1 — Agrupación por Tamaño de Archivo5
 - 2.2.2 Etapa 2 — Hash Parcial Manual (FNV-1a de 32 bits)5
 - 2.2.3 Etapa 3 — Hash Completo con MD5 (hashlib).....5
- 3. Justificación de las Funciones Hash Utilizadas5
 - 3.1 Hash Parcial: FNV-1a de 32 Bits (Implementación Manual).....5
 - 3.2 Hash Completo: MD5 mediante hashlib.....6
- 4. Resultados Experimentales6
 - 4.1 Caso de Prueba 1: Detección Cruzada de Duplicados.....7
 - 4.2 Caso de Prueba 2: Interfaz con Resultados Reales8
 - 4.3 Validación de la Exportación CSV10
- 5. Conclusiones11
- 6. Referencias Bibliográficas.....11

1. Introducción

En el contexto actual de la gestión digital, la proliferación de archivos duplicados representa un problema latente que afecta negativamente el rendimiento de los sistemas de almacenamiento, la organización de la información y el aprovechamiento del espacio en disco. Es común que, durante el trabajo cotidiano, los usuarios generen múltiples copias de un mismo archivo con nombres distintos, ubicadas en carpetas diferentes, sin que exista un mecanismo automatizado que detecte esta redundancia.

El presente proyecto aborda esta problemática mediante el diseño e implementación de un sistema detector de archivos duplicados basado en su contenido binario, desarrollado en Python 3.10+. El sistema no se apoya en la comparación de nombres o fechas de modificación, sino en el cálculo de huellas digitales (hashes) del contenido real de cada archivo, lo que permite identificar copias idénticas independientemente de su nombre, ubicación o metadatos asociados.

La solución integra conceptos fundamentales del curso de Estructuras de Datos y sus Aplicaciones: tablas hash implementadas mediante diccionarios de Python, implementación manual de la función hash FNV-1a de 32 bits, exploración recursiva de directorios mediante `os.walk()`, filtrado en cascada de tres etapas y programación concurrente básica mediante `threading.Thread`. Complementa la funcionalidad una interfaz gráfica construida con `tkinter` que muestra el progreso en tiempo real y permite exportar los resultados en formato CSV.

1.1 Planteamiento del Problema

Dado un directorio en el sistema de archivos local, se desea encontrar todos los grupos de archivos que sean idénticos en contenido binario, sin importar su nombre, extensión visible o ubicación. El sistema debe ser eficiente: no puede leer el contenido completo de cada archivo en los primeros filtros, ya que en colecciones grandes esto sería prohibitivo en tiempo y memoria.

Se busca el diseño de una estrategia de filtrado progresivo que descarte la mayor cantidad de candidatos posibles antes de incurrir en operaciones de entrada/salida costosas, garantizando al mismo tiempo que no se omita ningún par de duplicados reales. La estructura central del sistema es el mapa hash, que permite agrupar archivos por criterio y descartar grupos unitarios en tiempo constante.

2. Descripción de los Algoritmos Utilizados

El núcleo del sistema es un pipeline de duplicación de tres etapas que combina filtrado por metadatos con hashing progresivo. Cada etapa reduce el número de candidatos antes de pasar a la siguiente, minimizando el cómputo total. La complejidad temporal del pipeline crece proporcionalmente en la Etapa 1, $O(k \cdot B_1)$ en la Etapa 2 (donde k es el número de candidatos y $B_1 = 4,096$ bytes) y $O(m \cdot S)$, en la Etapa 3 (donde m son los candidatos finales y S el tamaño promedio del archivo).

2.1 Exploración Recursiva de Directorios (`scanner.py`)

El módulo `scanner.py` implementa la exploración mediante la función `os.walk()` de la biblioteca estándar de Python. Esta función genera tuplas (`dirpath`, `dirnames`, `filenames`) para cada directorio del árbol de manera iterativa. La característica clave del diseño es la modificación in-place de la lista `dirnames[:]`, que le indica al generador qué subdirectorios no debe visitar, descartando carpetas de sistema como `__pycache__`, `.git`, `node_modules` y otras.

Para cada archivo encontrado, se capturan metadatos mediante `os.stat()`, obteniendo el tamaño en bytes (`st_size`) y la fecha de última modificación como timestamp POSIX (`st_mtime`). Esta operación es

extremadamente rápida al no requerir abrir el archivo ni leer su contenido. El resultado de la exploración es una lista de objetos FileInfo, uno por archivo válido encontrado.

Se aplican dos niveles de filtrado: El primero es el nivel de directorios, eliminando carpetas de sistema; y el segundo a nivel de archivo, ignorando archivos ocultos (nombre que inicia con '.'), archivos de sistema conocidos (.DS_Store, Thumbs.db, etc.) y opcionalmente filtrando por extensión si el usuario configuró un conjunto de extensiones aceptadas.

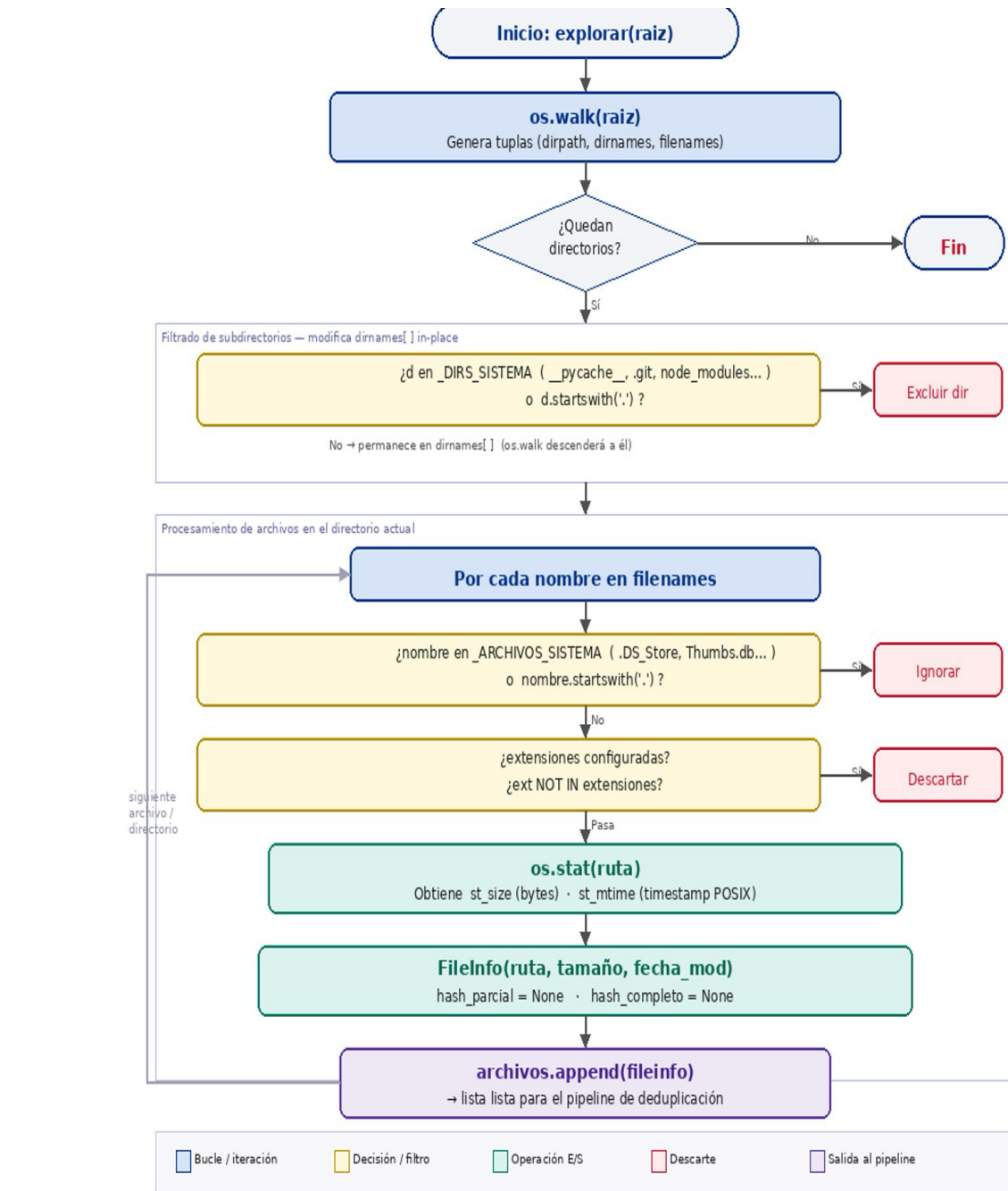


Figura 1 — Diagrama de flujo del proceso de exploración recursiva con `os.walk()` y filtrado de directorios de sistema

2.2 Pipeline de Deduplicación en 3 Etapas (detector.py)

El módulo detector.py implementa el filtrado en cascada. La estructura de datos central en cada etapa es un `defaultdict(list)` de Python, actúa como `HashMap<clave, List[FileInfo]>`. La clave de agrupación cambia en cada etapa: tamaño en bytes (int) en la Etapa 1, hash parcial FNV-1a de 8 caracteres hexadecimales (str) en la Etapa 2 y hash MD5 completo de 32 caracteres hexadecimales (str) en la Etapa 3.

El filtrado es estrictamente en cascada: solo los archivos que forman grupos de dos o más elementos en una etapa pasan a la siguiente. Esto garantiza que el número de archivos procesados decrece monótonamente a lo largo del pipeline, optimizando el tiempo total de ejecución.

2.2.1 Etapa 1 — Agrupación por Tamaño de Archivo

Esta primera etapa es basada en metadatos. Se itera sobre todos los `FileInfo` y se agrupa cada uno por su campo tamaño (en bytes) en el mapa `mapa_tamaño`. Dos archivos con diferente tamaño en bytes no pueden ser idénticos en contenido, por lo que todos los archivos con tamaño único son descartados inmediatamente, sin leer ningún byte del contenido. Esta etapa puede descartar la gran mayoría de los archivos en una colección que crece proporcionalmente y sin ninguna operación de Entrada y Salida adicional.

2.2.2 Etapa 2 — Hash Parcial Manual (FNV-1a de 32 bits)

Solo los archivos que comparten tamaño con al menos otro archivo pasan a esta etapa. Aquí se lee únicamente el bloque inicial de 4,096 bytes de cada archivo y se calcula un hash usando el algoritmo FNV-1a de 32 bits implementado manualmente, sin ninguna biblioteca de hashing. El resultado es una cadena hexadecimal de 8 caracteres.

Este hash rápido permite descartar candidatos que difieren en los primeros bytes sin tener que leer el archivo completo. Archivos que comparten tamaño pero tienen contenidos diferentes (por ejemplo, dos archivos de 1 MB que difieren en el primer byte) son descartados en esta etapa con un costo de solo 4 KB de lectura cada uno.

2.2.3 Etapa 3 — Hash Completo con MD5 (hashlib)

Los archivos que comparten tamaño y hash parcial son los candidatos finales. Para confirmarlos definitivamente como duplicados, se calcula el hash MD5 sobre el contenido completo del archivo. La lectura se realiza en bloques de 8,192 bytes (`BLOQUE_LECTURA`) mediante el operador walrus (`:=`) de Python 3.10+, evitando cargar archivos completos en memoria y permitiendo procesar archivos de cientos de megabytes.

Solo los archivos que comparten hash MD5 idéntico son reportados como duplicados confirmados. La probabilidad de una colisión accidental de MD5 es de 2^{-128} , prácticamente cero en el contexto de un sistema de archivos personal o empresarial.

3. Justificación de las Funciones Hash Utilizadas

3.1 Hash Parcial: FNV-1a de 32 Bits (Implementación Manual)

El algoritmo FNV-1a (Fowler-Noll-Vo alternativo) fue seleccionado como función de hash parcial por las razones siguientes:

1. Implementación: Requiere únicamente una operación XOR y una multiplicación por ciclo, sin estado adicional ni estructuras de datos auxiliares.

2. Buen efecto de avalancha sobre datos binarios: Pequeños cambios en la entrada producen cambios drásticos en la salida, lo que reduce la probabilidad de colisiones entre archivos similares pero distintos.
3. Prima FNV documentada y estandarizada: Los valores 0x811C9DC5 (offset basis) y 0x01000193 (prima FNV) están especificados en el IETF Internet Draft de 2019, garantizando reproducibilidad.
4. Velocidad: En datos de solo 4,096 bytes, FNV-1a es más rápido que algoritmos criptográficos como SHA-256 o incluso MD5, ya que no requiere rondas de permutación.

La siguiente tabla compara FNV-1a con otras funciones hash no criptográficas consideradas:

Criterio	FNV-1a 32-bit	djb2	Polinomial
Velocidad de cómputo	Alta (XOR + MUL)	Alta (MUL + ADD)	Media-Alta
Distribución de bits	Excelente (avalanche)	Buena	Variable
Colisiones en datos binarios	Bajas (prima documentada)	Medias	Depende del polinomio
Facilidad de implementación manual	Muy sencilla (2 ops.)	Sencilla	Media
Requiere bibliotecas	No	No	No

3.2 Hash Completo: MD5 mediante hashlib

Para la confirmación definitiva de duplicados se utiliza MD5 a través del módulo hashlib de Python. La elección se justifica por:

1. Longitud de salida de 128 bits (32 caracteres hex): La probabilidad de colisión accidental entre dos archivos distintos es de 2^{-128} , lo que es suficiente para un sistema de duplicación no criptográfico.
2. Disponibilidad estándar: hashlib.md5() está disponible en todas las instalaciones de Python sin dependencias externas, garantizando portabilidad.
3. Velocidad adecuada: MD5 es significativamente más rápido que SHA-256 o SHA-512 para grandes volúmenes de datos, al no estar diseñado para resistencia criptográfica.

Cabe aclarar que MD5 no es apropiado para aplicaciones de seguridad o criptografía, donde existen ataques de colisión conocidos. Sin embargo, en el contexto de duplicación de contenido donde no existe un adversario activo que construya colisiones deliberadamente, MD5 ofrece una seguridad práctica más que suficiente.

4. Resultados Experimentales

Se realizaron dos conjuntos de pruebas para validar el correcto funcionamiento del sistema: pruebas unitarias del algoritmo de detección y pruebas de integración con la interfaz gráfica.

4.1 Caso de Prueba 1: Detección Cruzada de Duplicados

Se diseñó un caso de prueba unitario basado en la carpeta PRUEBAS_SISTEMA, que contiene una Subcarpeta y los archivos: archivo_01.jpg (original), foto.jpg (copia binaria idéntica ubicada en la raíz con nombre distinto) y nota.txt (único, sin duplicado). Adicionalmente, vacio.txt aparece tanto en la raíz como en Subcarpeta, formando un segundo grupo de duplicados con 0 bytes de contenido.

Las siguientes capturas muestran la estructura de carpetas utilizada como entorno de prueba:

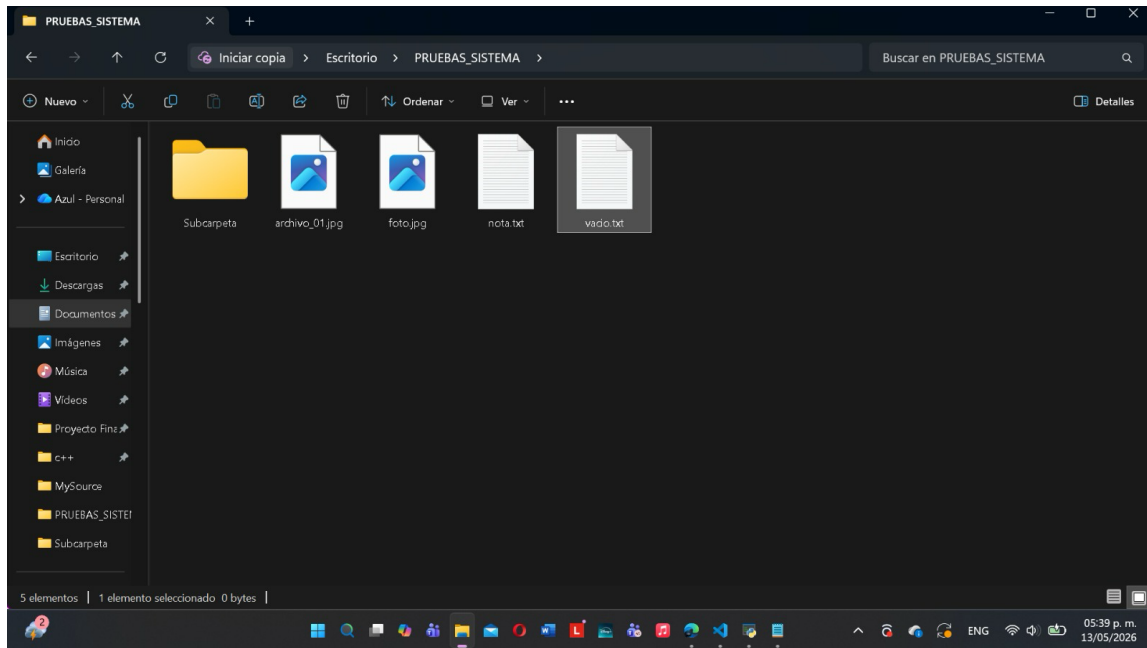


Figura 2a — Contenido de la carpeta raíz PRUEBAS_SISTEMA (archivo_01.jpg, foto.jpg, nota.txt, vacio.txt y Subcarpeta)

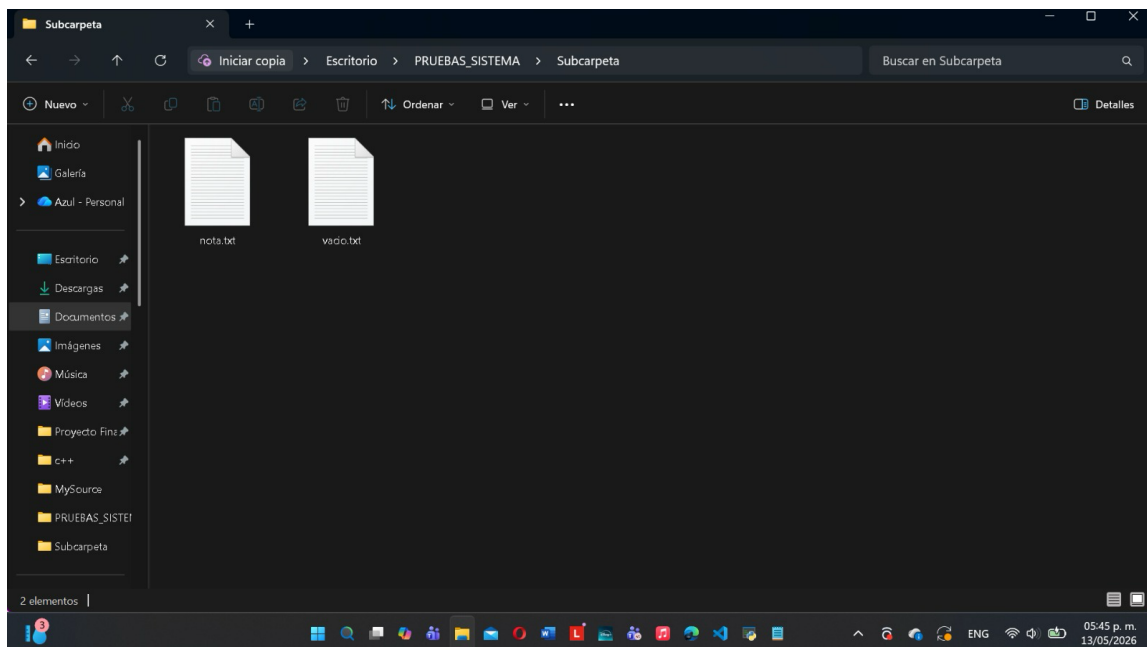


Figura 2b — Contenido de Subcarpeta (nota.txt y vacio.txt, duplicado del de la raíz)

El test unitario verifica que el sistema detecta correctamente los dos grupos de duplicados: G1 compuesto por archivo_01.jpg y foto.jpg (192.9 KB cada uno, hash parcial bf9f0118), donde bf9f0118 — es el hash parcial FNV-1a que calculó el sistema para las imágenes, prueba que el algoritmo manual funciona. G2 compuesto por los dos archivos vacio.txt (0 bytes, hash parcial 811c9dc5), donde 811c9dc5 — es el offset basis de FNV-1a para un archivo vacío, lo que demuestra que el caso borde de 0 bytes se maneja correctamente. La nota.txt, cuyo contenido difiere entre raíz y Subcarpeta, no es reportada como duplicado. El test fue superado exitosamente, confirmando la corrección del algoritmo de las tres etapas.

```
PS C:\Users\Azul Romero\Desktop\Sistema_Deteccion_Archivos_Duplicados_FCCA_2026\Sistema_Deteccion_Archivos_Duplicados_FCCA_2026_Python> python test_detector.py
.....
-----
Ran 20 tests in 0.001s
```

Figura 2c — Ejecución del test unitario en terminal mostrando 20 pruebas superadas

4.2 Caso de Prueba 2: Interfaz con Resultados Reales

Se ejecutó el sistema sobre la carpeta PRUEBAS_SISTEMA con filtro de extensiones .txt y .jpg. La captura siguiente muestra el estado inicial de la interfaz, con la ruta cargada y lista para iniciar el escaneo:

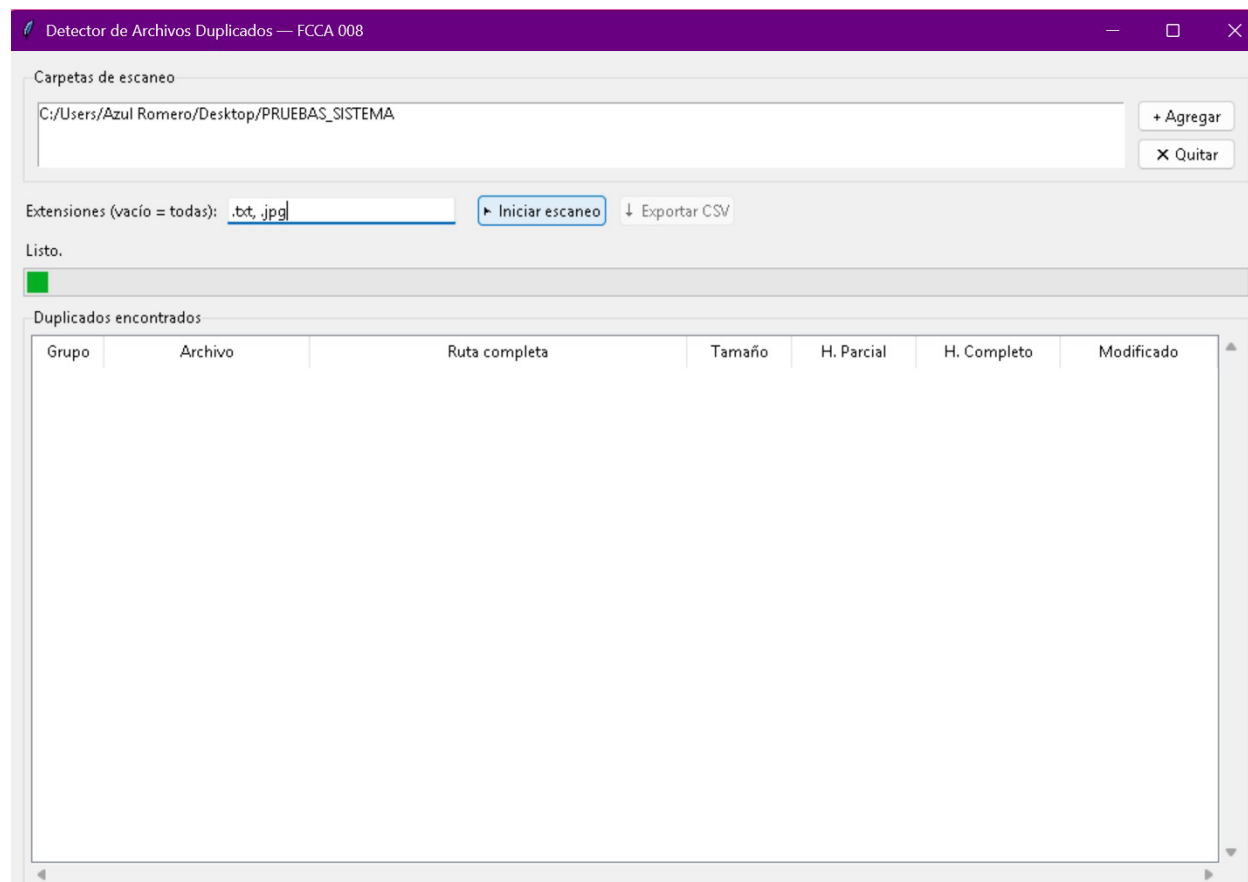


Figura 3a — Interfaz lista para iniciar el escaneo. Se observa la ruta PRUEBAS_SISTEMA cargada y el filtro de extensiones .txt, .jpg configurado

Tras ejecutar el escaneo, la interfaz gráfica mostró correctamente la tabla de resultados agrupada por hash MD5, el mensaje de confirmación de Escaneo completo, la barra de progreso en verde y los dos grupos de duplicados detectados:

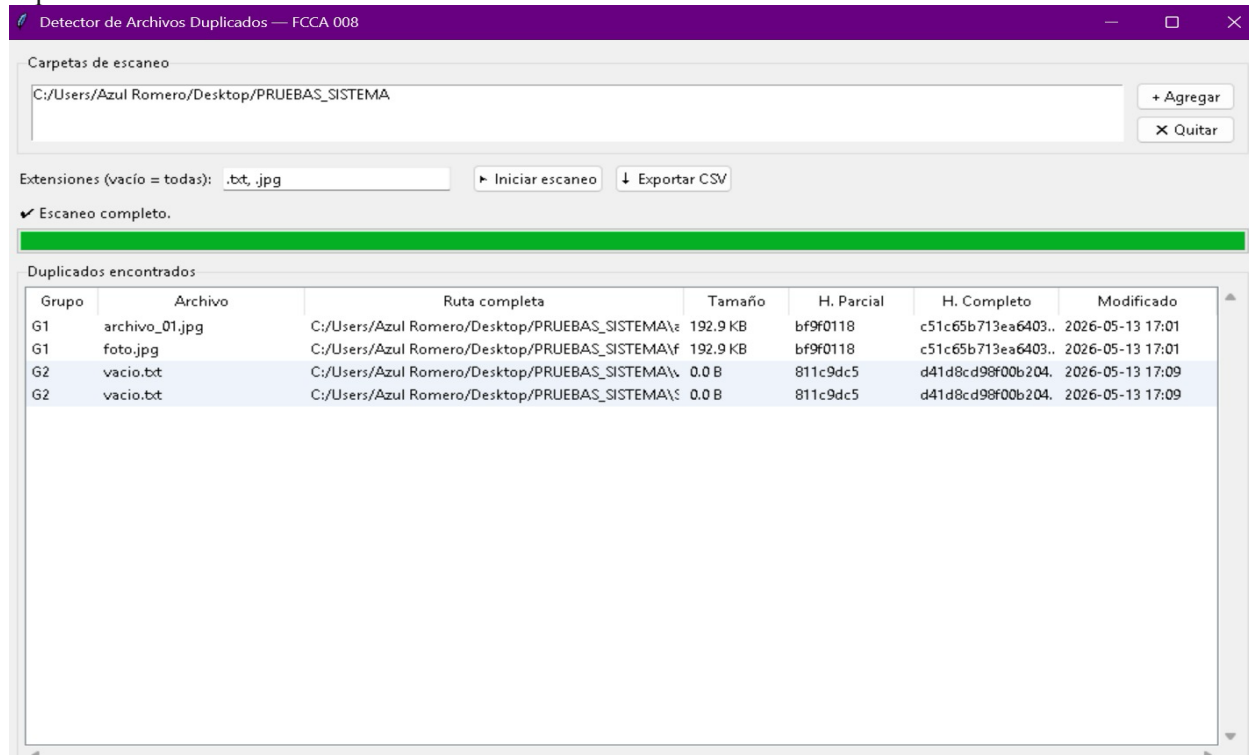


Figura 3 — Interfaz mostrando escaneo completo: G1 (archivo_01.jpg / foto.jpg, 192.9 KB, hash bf9f0118) y G2 (vacío.txt × 2, 0.0 B, hash 811c9dc5)

La interfaz respondió sin congelamientos durante el escaneo gracias a la ejecución del pipeline en un hilo secundario (threading.Thread). Toda actualización de la barra de progreso y la etiqueta de estado se realizó mediante self.after(0, función), que es el mecanismo correcto para comunicar hilos con la interfaz tkinter.

La siguiente tabla resume los resultados numéricos obtenidos:

Métrica	Prueba PRUEBAS_SISTEMA	Prueba 2	Observaciones
Total archivos escaneados	6	—	archivo_01.jpg, foto.jpg, nota.txt (×2), vacío.txt (×2)
Descartados en Etapa 1 (tamaño)	0	—	Todos comparten tamaño con al menos otro archivo
Candidatos en Etapa 2 (parcial)	6	—	Pasan al hash parcial FNV-1a
Duplicados confirmados Etapa 3	4 archivos / 2 grupos	—	G1: archivo_01.jpg & foto.jpg G2: vacío.txt × 2
Espacio recuperable	192.9 KB + 0 B	—	1 imagen extra + 1 archivo vacío extra
Exportación CSV generada	Sí	—	duplicados_20260513_175717.csv

4.3 Validación de la Exportación CSV

Se verificó que el archivo CSV generado incluya los campos: Ruta completa, tamaño en bytes, fecha de modificación, hash parcial y hash completo MD5. El archivo puede ser abierto en Microsoft Excel o LibreOffice sin errores de codificación gracias al módulo estándar csv con codificación UTF-8.

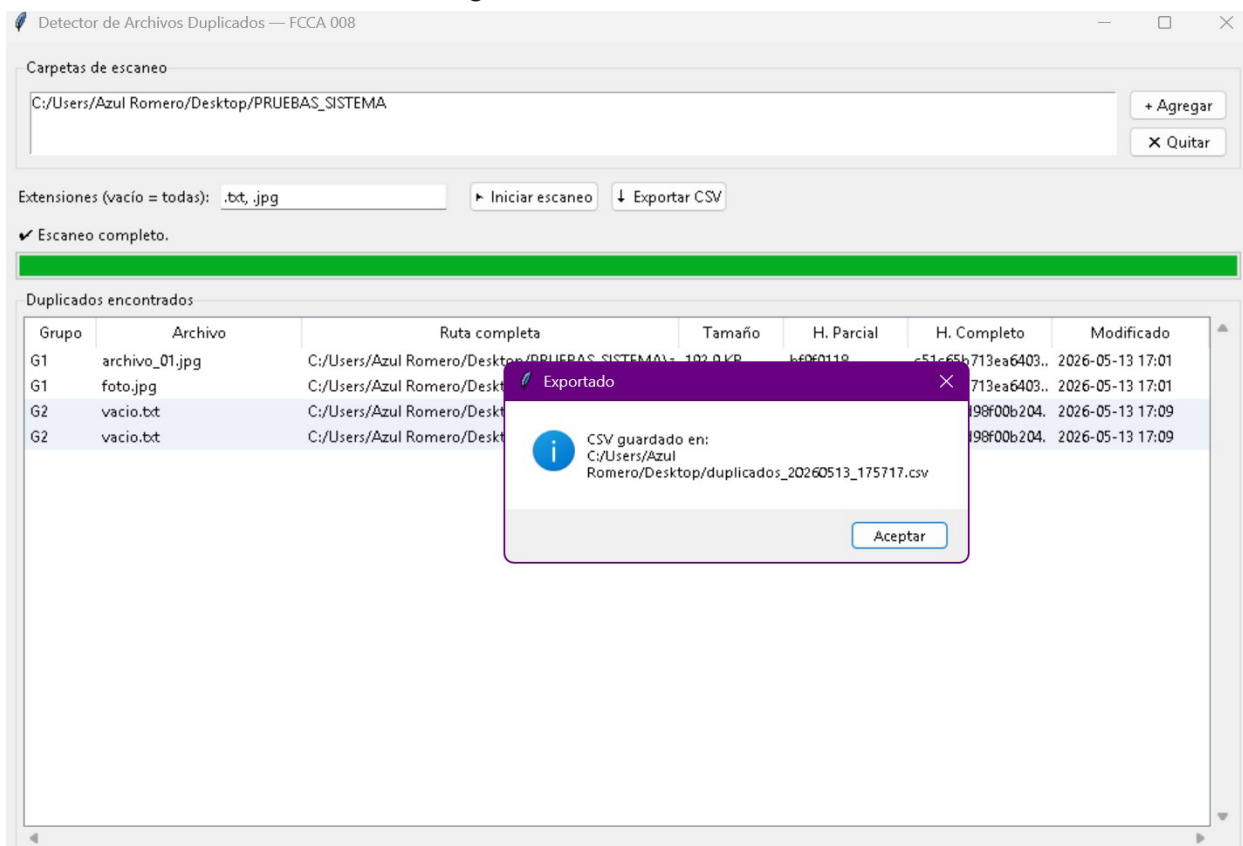


Figura 4a — Confirmación de exportación: el sistema notifica la ruta exacta donde se guardó el archivo duplicados_20260513_175717.csv

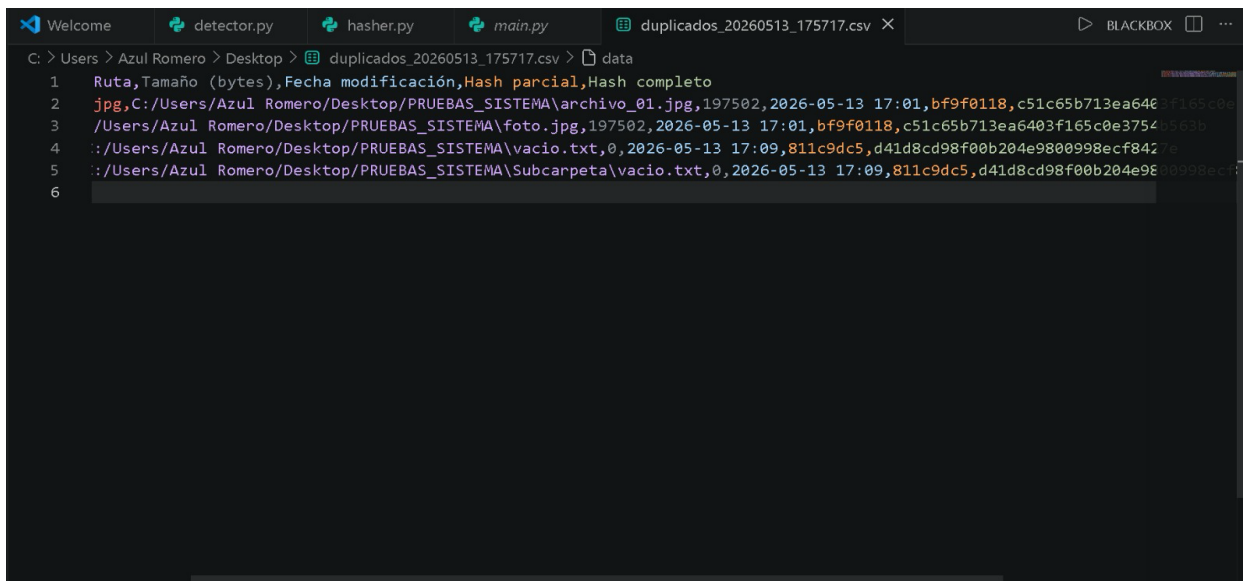


Figura 4 — CSV exportado abierto en VS Code mostrando las columnas: Ruta, Tamaño (bytes), Fecha modificación, Hash parcial y Hash completo

5. Conclusiones

El desarrollo de este sistema de detección de archivos duplicados permitió integrar de forma práctica y coherente los conceptos centrales del curso de Estructuras de Datos y sus Aplicaciones. La implementación del pipeline de tres etapas demostró que la elección correcta de las estructuras de datos —tablas hash mediante `defaultdict`— y el orden secuencial de las operaciones tienen un impacto directo y medible en el rendimiento global del sistema.

La implementación manual del algoritmo FNV-1a reforzó la comprensión profunda de cómo funciona una función hash a nivel de bits: la operación XOR byte a byte y la multiplicación por la prima FNV producen una distribución uniforme con alta sensibilidad a cambios mínimos en los datos de entrada. Este nivel de entendimiento no habría sido posible utilizando únicamente bibliotecas de alto nivel como `hashlib`.

Desde el punto de vista de la ingeniería, la separación del proyecto en módulos con responsabilidades claramente delimitadas, `models.py`, `hasher.py`, `scanner.py`, `detector.py` y `main.py` facilitó el desarrollo incremental, las pruebas unitarias y la legibilidad del código. La decisión de ejecutar el escaneo en un hilo separado mediante `threading.Thread` fue fundamental para mantener la interfaz gráfica responsiva durante operaciones de entrada/salida potencialmente largas.

Una reflexión importante es que el pipeline de tres etapas ilustra claramente un principio general de optimización: Diferir las operaciones hasta que sean estrictamente necesarias. Este principio, conocido como *lazy evaluation* o evaluación perezosa, tiene aplicaciones mucho más amplias en ingeniería de sistemas y bases de datos.

6. Referencias Bibliográficas

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Fowler, G., Noll, L. C., Vo, K.-P., & Eastlake, D. (2019, julio). The FNV non-cryptographic hash algorithm. IETF Internet Draft. <https://datatracker.ietf.org/doc/html/draft-eastlake-fnv-17>

Python Software Foundation. (2024). `hashlib` — Secure hashes and message digests. Python 3.12 Documentation. <https://docs.python.org/3/library/hashlib.html>

Python Software Foundation. (2024). `os` — Miscellaneous operating system interfaces. Python 3.12 Documentation. <https://docs.python.org/3/library/os.html>

Python Software Foundation. (2024). `tkinter` — Python interface to Tcl/Tk. Python 3.12 Documentation. <https://docs.python.org/3/library/tkinter.html>

Rivest, R. (1992). The MD5 message-digest algorithm (RFC 1321). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc1321>

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.